WARWICK TRADING SOCIETY

QUANT DIVISION

INTRASOC / PARTICIPANT HANDBOOK

Market Making Challenge

Build a Python trading bot, manage inventory and compete for net profit across three assets.

03	10,000	PYTHON
TRADING ASSETS	DECISIONS / MATCH	BOT LANGUAGE

Updated 11 November 2026

PARTICIPANT EDITION

The challenge

Build a market-making bot that chooses where to bid, where to ask and how much inventory to hold at every decision tick.

A · B · C	±50	20
SEPARATE BOOKS	POSITION / ASSET	MAX UNITS / SIDE

The objective

Earn the highest **net profit and loss (P&L)** across the three assets. Competitors share an order book within each asset; an order in A cannot fill against B or C. Separate order books do not imply statistically independent price movements.

What a market maker does

Bid	Your offer to buy at a specified price and size.
Ask	Your offer to sell at a specified price and size.
Spread	Ask minus bid. A wider spread pays more per completed round trip, but may attract fewer fills.

Worked example: both sides fill

Buy	Your bid fills 5 units at 99.98, costing 499.90 before fees.
Sell	Your ask fills 5 units at 100.02, receiving 500.10 before fees.
Result	Flat inventory; 5 × (100.02 − 99.98) = **0.20 gross P&L**. Fees reduce this where applicable.

THE CATCH	Only one side may fill. Buying five without selling them leaves you long five units, exposed to the eventual closing valuation.

What your bot observes

The engine supplies an Observation object every tick. It contains the tick, your cash and a read-only market view for each asset.

```
observation.tick          # Current decision time
observation.cash          # Your cash across assets
observation.markets["A"]  # Asset A public view + own state
observation.markets["B"]  # Asset B public view + own state
observation.markets["C"]  # Asset C public view + own state
```

Understanding markets["A"]

It is an **AssetObservation**, not a trading engine or a forecast. The market-level observations can be used by all bots; **position** and **recent_fills** are specific to your bot.

price_history	Tuple of past public reference prices, initially 100. Some entries may be None.
book	Up to five aggregated price levels on each side of the visible order book.
position	Your signed inventory: e.g. +12 is long 12; -12 is short 12.
recent_fills	Your fills since the last decision: time, side, price, quantity and own order ID.

How to read the visible book

```
market = observation.markets["A"]
example_book = {  # illustration, not live data
    "best_bid": 99.98, "best_ask": 100.02, "mid": 100.00,
    "bids": [{"price": 99.98, "quantity": 25}],
    "asks": [{"price": 100.02, "quantity": 30}]
}
best_bid = market.book["best_bid"]
```

The **book midpoint** comes from the visible best bid and ask; it can differ from **price_history**. A missing side means best bid, best ask or midpoint may be **None**.

Matching and trading rules

Your bot submits one joint decision for all three assets. Its quotes are checked, inserted and matched against eligible interest.

What happens in a tick

1. Observe	Receive the last completed public history/book, your inventory, cash and recent fills.
2. Quote	Return QuoteAction objects for any assets you choose to trade.
3. Validate	Check prices and sizes; valid sizes may be clipped to order and position limits.
4. Match	Orders match at the resting quote price, prioritising price and then time.
5. Update	Cash and inventory change. Recent fills are delivered at your next decision; old quotes are replaced.

Order and position constraints

Position	−50 to +50 units per bot, per asset.
Size	At most 20 units per quote side; nonnegative integers.
Prices	Positive, finite and on the 0.01 grid; bid strictly below ask when both are active.
One-sided	A zero-sized side is inactive; omitting an asset withdraws its old quotes.

Order priority

At a price level, earlier resting orders are served first. Bot processing time affects insertion order for same-tick submissions: runtime is measured on the organiser system, grouped into 0.1 ms buckets, then ties use accepted size, bot ID and side. A quote crossing the visible opposing best price can trade immediately.

REMEMBER	Assets have separate order books. Other bots compete with you for fills **within** each asset during head-to-head matches.

Assets, fees and inventory

Each asset uses the same trading interface but has its own account position and transaction fee.

100.00	10,000	0.01
INITIAL PRICE / ASSET	DECISION TICKS	MINIMUM PRICE TICK

Fees on every execution

Asset A	0.000 per traded unit
Asset B	0.005 per traded unit
Asset C	0.010 per traded unit

Fees are charged on every buy and sell, not just on round trips. A completed five-unit round trip in B trades ten units in total, so its fee is 10 × 0.005 = 0.05.

Why inventory matters

If your position in Asset A is +48, buying more is nearly impossible under the +50 limit, but you have ample room to sell. The engine clips valid quotes to available position headroom. A flat position means zero units; a negative position means short.

Cash is not profit

Buys reduce cash and increase position. Sells increase cash and reduce position. An apparently large positive cash balance can coexist with a loss-making short position. The final mark-to-market calculation values all remaining inventory.

QUOTING TIP You may widen a spread, reduce size, skew toward unwinding inventory or stop quoting one side. A competitive bot balances spread earned against the risk from unequal fills.

Code your first bot

Submit one Python file containing a concrete bot class. It must have a no-argument constructor and implement get_quotes(observation, memory).

Required return: (quotes, memory), where quotes maps asset IDs to QuoteAction(bid_price, bid_size, ask_price, ask_size). Memory starts as {} and persists between ticks of a match.

```
import math
from bots.base import BaseBot
from models.models import QuoteAction

class StarterBot(BaseBot):
    def get_quotes(self, observation, memory):
        quotes = {}
        tick_size = 0.01
        for asset, market in observation.markets.items():
            ref = next((p for p in reversed(market.price_history)
                        if p is not None and math.isfinite(p)
                        and p > 0), None)
            if ref is None:
                continue
            bid_tick = max(1, math.floor(ref / tick_size - 3))
            ask_tick = max(bid_tick + 1,
                           math.ceil(ref / tick_size + 3))
            bid_size = max(0, min(5, 50 - market.position))
            ask_size = max(0, min(5, 50 + market.position))
            quotes[asset] = QuoteAction(
                bid_price=round(bid_tick * tick_size, 12)
                          if bid_size else None,
                bid_size=bid_size,
                ask_price=round(ask_tick * tick_size, 12)
                          if ask_size else None,
                ask_size=ask_size,
            )
        return quotes, memory
```

This quotes five units on each side of the last usable public reference, respecting the current position. It is a **baseline**, not a forecast or a guaranteed profitable strategy.

Practice with the backtester

You will receive a separate participant repository containing public training data and an approximate backtester.

What you will receive

Backtester	Runs your bot on supplied data to estimate fills, inventory and P&L.
events.csv	Historical public order, cancellation, timing, price and size records for each asset.
market_info.json	Published initial prices, tick grids, limits, fees and horizon.
README.md	Installation, backtest commands, and report instructions in the participant repository.

START HERE Follow the **README.md in the supplied participant repository** to install dependencies, run practice backtests and interpret your results. You do not need the organiser-only repository.

Backtesting versus head-to-head

The practice backtester **roughly approximates** your bot's behaviour but runs without direct competition from other submitted strategies. In the tournament, bots share order books within each asset and compete for queue position and executions. Practice performance is likely to be an easier baseline.

Research with the training tape

```
import pandas as pd

events = pd.read_csv("events.csv")
a = events[events["asset_id"] == "A"]
print(a[["tick", "kind", "side", "price"]].head())
```

No look-ahead: the CSV covers complete past scenarios for offline research. At decision tick t your strategy must only use information available through t, not future order events.

Scoring and results

The primary performance measure is final net P&L, aggregated across Assets A, B and C, after execution fees.

Official terminal P&L

```
net P&L = final cash
        + sum(final position[a] * closing mark[a]
              for each asset a)
        - initial cash
```

Open positions are marked at the official match closing valuations. Those marks are not part of the observation given to your bot. Public P&L charts instead value positions using available public reference prices, so their endpoints can differ from the official result.

Example: net spread after fees

Buying five units of Asset B at 99.98 and selling five at 100.02 earns **0.20 gross spread**. Fees on ten traded units total 0.05, leaving **0.15 net P&L** if inventory finishes flat and there are no other trades.

What your public report shows

Official P&L	Your terminal match result, net of fees and including remaining inventory.
Public equity	A diagnostic P&L curve using published reference marks, with per-asset views.
Fills and orders	Your own quote history, executed trades, fees, sizes and cash changes.
Risk and validity	Inventory, drawdown, accepted/clipped sizes and quote errors.
Runtime	Median and high-percentile decision latencies.

MATCH WINNER In a head-to-head match, higher official net P&L is the natural comparison. Tournament-wide aggregation and tie-break rules will be published separately.

From working to competitive

Get a valid bot running, check its behaviour, and only then experiment with more sophisticated quoting rules.

Three experiments

Inventory skew	Shift bid/ask prices to encourage fills that reduce a large position.
Dynamic spread	Widen or narrow quotes based on observable prices or book liquidity.
Adaptive size	Decrease exposure when your inventory or observed market conditions become difficult.

Submission checklist

- A single **.py** file with one concrete strategy class, or an explicitly named class.
- A no-argument constructor and **get_quotes(observation, memory)**.
- Return an asset-to-**QuoteAction** dictionary and your updated memory each tick.
- Support all assets in **observation.markets** and handle missing or empty books.
- Keep prices positive, finite and on the 0.01 grid, with integer order sizes.
- Respect ±50 inventory and 20-unit order caps, including one-sided quoting.
- Test using the participant repository and examine net P&L after fees.

> **FINAL CHECK** Use the participant repository **README.md** for the latest run and submission instructions. Do not assume a strategy that works in standalone backtests will perform equally well against competing bots.

Mini glossary

Fill	An executed quantity on one of your orders.
Inventory	Your signed units currently held in an asset.
Reference	A public price series distinct from the live book midpoint.